

Homework8 参考答案

Problem 1

1) 用 2KB 管理 16384 个磁盘块的空闲状态

2KB = 2048 字节 = 16384 位，恰好每个磁盘块 1 位。所以用位图 (bitmap) 管理：每个块对应 1 个 bit，0 表示空闲、1 表示已用（或相反）。分配/释放即翻转对应位，查空闲即扫描位图。

2) C-SCAN 读 4 个扇区的总时间

磁盘参数：6000 RPM，每磁道 100 扇区，相邻磁道平均移动 1ms。当前磁头在 100，向磁道号增大方向移动。请求队列 50、90、30、120，每个请求读一个随机分布的扇区。

转速换算：

```
一圈时间 = 60s / 6000 = 10ms  
平均旋转延迟 = 半圈 = 5ms  
读 1 个扇区(传输) = 10ms / 100 = 0.1ms
```

C-SCAN（按课件定义）：磁头沿增大方向移动到当前请求中最内（号最大）的请求，再置位到当前请求中最外（号最小）的请求，继续沿增大方向服务。服务顺序：120，（置位回到 30），30，50，90。

寻道距离（每相邻磁道 1ms）：

```
100 -> 120 : 20  
120 -> 30 : 90 (置位到最外侧请求 30)  
30 -> 50 : 20  
50 -> 90 : 40  
寻道合计 = 20 + 90 + 20 + 40 = 170 个磁道 = 170ms
```

每个扇区的旋转+传输：

```
4 * (旋转 5ms + 传输 0.1ms) = 4 * 5.1ms = 20.4ms
```

总时间 = 170 + 20.4 = 190.4ms。

3) 换成随机访问的 Flash 设备后，什么比 C-SCAN 更有效

Flash/SSD 随机访问没有机械寻道和旋转延迟，访问任意位置代价基本一致，C-SCAN 这种“按磁道顺序优化寻道”的调度失去意义。更有效的做法是不做寻道排序，用 FCFS / Noop 即可；并转而利用 SSD 的内部并行性（多通道 / 多 chip / 多 plane 并行）来调度，同时考虑写放大与磨损均衡 (wear leveling)，而不是优化访问顺序。

Problem 2

本题原文那句“假设更新 page 都是连续写入一个 block 的，不论更新 page 是来自于哪个 block”描述的是 FAST (fully associative sector translation) 算法，而非笔误；但题面缺少 FAST 必需的参数 (log block 上限、victim 规则、块数是否无限)，需按严格版本作答，详见 作业/Homework8/Homework8-题目勘误.md。下面按严格版本求解：

1. 更新一律连续追加写入 fully-associative 的 log block，不论来自哪个数据块。
2. 同时最多 2 个 log block；当前 log block 写满后未达上限则新开一个，达上限且仍需空间则按 round-robin(先进先出)选最早开启者作 victim 回收。
3. 回收 victim 时，对其中含有效页的每个数据块各做一次合并：把该块全部最新页(可能散落在 victim、其他 log block、旧数据块)收集到新块、擦旧数据块；victim 涉及的数据块都合并完后擦除 victim。
4. 空白 block 用 first-fit；空闲块数量充足(不限于图中 4 个)。

初始：block table 1000→block0、2000→block1 (即 2000→物理页 4)。block0=a,b,c,d，block1=e,f,g,h，其余块空，log table 空。偏移：a/e=0, b/f=1, c/g=2, d/h=3。更新序列：e, f, a, a, a, g, g, f, h, e。下面逐步模拟，箭头后为该次写入的物理页号，LB-A/LB-B/LB-C 表示先后开启的 log block，round-robin 队列按开启顺序排列。

```

e  开 LB-A=block2(页8-11) 续写: e@8                log: e@8
f  续写 LB-A: f@9                log: e@8 f@9
a  续写 LB-A: a@10               log: e@8 f@9 a@10
a  续写 LB-A: a@11 (a@10失效) LB-A写满             log: e@8 f@9 a@11
a  LB-A满, 活跃1<2 -> 开 LB-B=block3(页12-15): a@12(a@11失效) log: e@8 f@9
a@12
g  续写 LB-B: g@13                log: e@8 f@9 a@12
g@13
g  续写 LB-B: g@14 (g@13失效)      log: e@8 f@9 a@12
g@14
f  续写 LB-B: f@15 (f@9失效) LB-B写满             log: e@8 a@12 g@14
f@15
h  两块都满且活跃=2上限 -> round-robin 取最早的 LB-A 作 victim 回收
    LB-A 有效页只剩 e@8, 涉及的数据块只有 2000
    对 2000 做 full merge: 收集最新 e@8(victim)、f@15(LB-B)、g@14(LB-B)、
    h(旧block1页7) 写入新块 block4(页16-19)=[e,f,g,h], block table
2000→16,
    擦旧数据块 block1;再擦 victim block2
    (LB-B 中 f@15、g@14 随之失效, LB-B 仅剩 a@12 有效)
    然后开 LB-C=block1(first-fit, 刚擦空, 页4-7): h@4 (block4 的 h@19 失
效)
                                                log: a@12 h@4
e  续写 LB-C: e@5 (block4 的 e@16 失效)           log: a@12 h@4 e@5
  
```

整个过程只在 h 这一步触发了一次回收，其中只对数据块 2000 做了一次 full merge。

四个小问:

1. 写放大（按课件“搬移 pages”口径）

full merge(2000): 收集 e,f,g,h 到 block4 = 4 page
写放大 = 4 个 page

2. 擦除次数

擦旧数据块 block1 (1次) + 擦 victim log block block2 (1次) = 2 次

3. 各类 merge 次数

switch merge : 0 次
partial merge: 0 次
full merge : 1 次

4. 序列完成后的系统状态

Block Table: 1000 -> block0, 2000 -> block4
Log Table: a -> 页12(block3), h -> 页4(block1), e -> 页5(block1)

block0 = 旧 1000: [a*, b , c , d] a* 失效(最新 a 在 log 页12), b/c/d 有效
block1 = LB-C: [h , e , - , -] h@4、e@5 有效
block2 = 空 (已擦除)
block3 = LB-B: [a , g*, g*, f*] a@12 有效, g@13/g@14/f@15 均已失效
block4 = 数据 2000:[e*, f , g , h*] f@17/g@18 有效, e@16/h@19 失效(最新在 log)

可见相比课件 BAST 模型（同一序列下约 3 次 merge、5 次擦除、写放大 10 page），FAST 凭借全关联 log block 把更新尽量积攒、推迟合并，这里只发生 1 次 full merge、2 次擦除、写放大 4 page，但代价是数据块 2000 的最新页被打散在多个 log block 中、合并时需要跨块收集。若“写放大”改用“只统计被白搬的未更新页”口径，本次 full merge 中只有 h 是未被更新过的页，则写放大记为 1 page（其余三问不变）。

Problem 3

```
int main() {
    int fd1, fd2;
    char c;
    fd1 = open("foo.txt", O_RDONLY, 0);    // fd1 = 3
    fd2 = open("bar.txt", O_RDONLY, 0);    // fd2 = 4
    close(fd2);                            // 关闭 4
    fd2 = open("bar.txt", O_RDONLY, 0);    // 重新打开，仍得 fd2 = 4，偏移从 0
    开始
```

```

int rc = fork();                                // 父子共享该打开文件描述（共享文件偏
移)
if (rc == 0) {                                  // 仅子进程
    read(fd2, &c, 1);
    printf("fd2=%d, child: c=%c\n", fd2, c);
}
read(fd2, &c, 1);                                // 父、子都执行
printf("fd2=%d, c=%c\n", fd2, c);              // 父、子都执行
exit(0);
}

```

文件内容：foo.txt = "foofoofoo\n", bar.txt = "barbarbar\n".

要点：

- fd2 始终是 4（关闭后重开拿到最小可用号 4，fd1=3 仍占用）。
- fork 后父子共享 fd2 对应的同一个打开文件描述，因此共享文件偏移。
- 一共发生 3 次 read（子进程 2 次：if 内 1 次 + if 后 1 次；父进程 if 后 1 次），它们在同一个偏移上推进，依次读到 bar.txt 的第 0、1、2 个字节，即 b、a、r。

设三次读为 C1（子 if 内）、C2（子 if 后）、P（父），约束 C1 在 C2 之前，P 任意穿插。按读发生的先后，谁先读谁拿到更靠前的字节，可能的取值组合为：

读顺序	C1	C2	P
C1, C2, P	b	a	r
C1, P, C2	b	r	a
P, C1, C2	a	r	b

对应的输出（fd2 都是 4；每个进程内部 "child:" 行在前、普通行在后，三行之间还可进一步交错）：

```

情形一 (b,a,r):
  fd2=4, child: c=b
  fd2=4, c=a           (子)
  fd2=4, c=r           (父)

情形二 (b,r,a):
  fd2=4, child: c=b
  fd2=4, c=r           (子)
  fd2=4, c=a           (父)

情形三 (a,r,b):
  fd2=4, child: c=a
  fd2=4, c=r           (子)
  fd2=4, c=b           (父)

```

也就是说：子进程 "child:" 行打印 b 或 a；父进程那行打印 r、a 或 b；三次读到的字节 恰好是 b、a、r 的一个分配。三行的相对打印顺序还可因调度而交错（只要保证同一进程内 "child:" 行先于其后的普通行）。